

Sensiq

Technical Reports: CL-2026-42, July 2026

Simon Völkl, Johannes Schieder, Sebastian Rosner and Andre Tien Vu 

Department of Electrical Engineering, Media, and Computer Science
Ostbayerische Technische Hochschule Amberg-Weiden
Amberg, Germany

Abstract—
Index Terms—

I. INTRODUCTION

Reliable indoor environment quality (IEQ) monitoring is a recurring requirement in laboratories, production halls, and similar facilities, where ambient conditions like indoor air quality (IAQ) directly affect experimental validity, product quality, and occupational safety. Existing Internet of Things (IoT) monitoring platforms address parts of this problem but typically rely on on-premises servers, offer only coarse sampling, or restrict access through proprietary applications (Section II).

This report describes Sensiq, a microcontroller unit (MCU)-based IoT architecture combining ESP32 (Espressif Systems) edge devices with a serverless Amazon Web Services (AWS) cloud backend. Sensor readings are transmitted securely over Message Queuing Telemetry Transport (MQTT) using Transport Layer Security (TLS), validated, and persisted for both live and historical views. Exposed through Representational State Transfer (REST) Application Programming Interface (API) endpoints, a React-based frontend visualises the data, while threshold-driven email alerts notify personnel of critical events.

A. Mission Statement

Sensiq aims to deliver an end-to-end, low-latency environmental monitoring system that provides both real-time visibility and cost-efficient historical analytics. By leveraging a fully serverless backend, it enables stakeholders to continuously track critical ambient parameters—such as temperature, humidity, light intensity, gas concentration, and flame exposure—to ensure safety and quality assurance.

B. Motivation

Unnoticed deviations in ambient conditions can invalidate experiments, damage sensitive samples, and endanger personnel [1, 2]. While continuous monitoring is essential, existing academic IoT platforms (e.g., [3, 4]) often rely on high-maintenance on-premises stacks and employ sampling intervals that are too coarse to capture short-lived events. Sensiq closes this gap by pairing inexpensive edge hardware with a scalable cloud architecture, keeping infrastructure and operating costs low enough for routine laboratory use. The full source code is publicly available under the MIT License [5].

C. Document Structure

The remainder of this report is organised as follows. Section II positions Sensiq against existing academic and commercial monitoring solutions and motivates the design decisions taken in this project. Section III introduces the conceptual building blocks of the system and presents the end-to-end architecture in a technology-agnostic manner.

The subsequent three sections refine this concept along the data flow: Section IV details the ESP32-based sensor node and its secure MQTT publication to AWS IoT Core; Section V describes the serverless cloud pipeline, covering both the low-latency live route and the long-term history route; and Section VI explains how end users consume the data through the React-based web frontend and how they are notified of critical events via email. Section VII assesses the resulting system with respect to testability, operating cost and a discussion. Finally, Section VIII summarises the contribution of the report and outlines directions for future work.

II. RELATED WORK

Several IoT-based platforms for indoor environmental monitoring have been proposed. This section contrasts two representative academic systems — by Mumtaz et al. [3] and Marques and Pitarma [4] — with the design decisions taken for Sensiq.

A. On-Premises IoT Air Quality Monitoring

Mumtaz et al. [3] present a multi-sensor node targeting air quality prediction. While sharing Sensiq’s general topology, their system differs fundamentally in three areas.

First, they rely on a local web server and database, tying the platform to on-premises operation. Sensiq instead targets a fully serverless cloud backend (Section V) to scale on demand and keep idle costs near zero.

Second, their five-minute sampling interval is geared towards trend analysis, which is too coarse for safety-relevant events like fires or for product quality assurance where short-lived deviations can compromise samples. Sensiq utilizes a low-latency MQTT-over-TLS pipeline to surface critical conditions immediately.

Third, their twenty-hour battery autonomy limits remote deployment. Sensiq’s ESP32 architecture could support deep sleep operation, as mentioned in Section VII.

B. iAQ+ Laboratory Environment Supervision System

Marques and Pitarma [4] propose iAQ+, an ESP8266-based monitoring system. While hardware choices are similar, significant differences exist in backend architecture, frontend access, and timing guarantees.

On the backend, iAQ+ employs a traditional server-based stack (SQL Server and .NET), which requires continuous hosting. Sensiq delegates persistence and routing to managed AWS services.

On the frontend, iAQ+ restricts access to iOS devices and limits the web dashboard to hourly aggregations. Sensiq provides platform-independent access via open REST endpoints, a React frontend, and a true real-time view (Section VI).

Finally, iAQ+ lacks explicit latency guarantees for threshold alerts. Sensiq is efficiently designed around instantaneous, threshold-driven email notifications.

III. TECHNICAL CONCEPT

High-level, technology-agnostic view of the system. This section introduces the conceptual building blocks and the end-to-end data flow. Detailed component-level descriptions follow in the Hardware, AWS Services, and User Interaction sections.

A. Technical Building Blocks

Enumerate the logical components of the system without binding them yet to concrete products: edge sensing node, secure transport channel, ingestion gateway, validation stage, hot-path store (live data), cold-path store (history), query API, notification channel, and user interface. Briefly justify the chosen programming languages (C++ firmware, Python lambdas, TypeScript infrastructure, React UI).

B. System Architecture

Present the architecture diagram (see template: fig:cloud-architecture) and walk the reader through the data flow: ESP32 -> MQTT/TLS -> AWS IoT Core -> validation Lambda -> dual persistence (DynamoDB for live, S3 + Athena for history) -> REST endpoints -> frontend / SNS.

IV. HARDWARE

The sensor node is the physical edge component of the Sensiq system, built around an ESP32 microcontroller with a set of environmental sensors. This section maps the hardware components (Section IV-A), outlines the firmware measurement loops (Section IV-B), and details the secure MQTT publication strategy (Section IV-C).

A. Component Overview

The sensor node is built using the following core components:

- **ESP32 NodeMCU:** Central microcontroller with integrated Wi-Fi and Bluetooth [6, 7].
- **DHT11:** Basic digital temperature and humidity sensor [8].
- **Joy-it KY-026 & KY-028:** Analog flame sensor (IR band) and NTC thermistor, both with adjustable digital thresholds [9, 10].

- **TSL2561:** Digital ambient-light sensor for visible and IR light [11, 12].
- **Bosch BME680:** Comprehensive sensor for precise temperature, humidity, pressure, and VOC gas concentrations [13–15].
- **Auxiliary Switches:** Two inputs reserved for labeling training data and outliers (see Section VIII).

The hardware design is documented using two complementary views. Figure 1 shows the logical circuit diagram (created with KiCAD [16]), detailing wiring and GPIO assignments. The functional prototype is shown in Figure 2, demonstrating the assembled sensor node.

B. Measurement Workflow

The firmware on the ESP32 follows a deterministic, two-phase lifecycle written in Arduino C++ [17]: a one-time *startup* phase that prepares the networking and sensing subsystems, followed by a continuously running *measurement loop*.

During startup the node performs the following steps in order:

- 1) Wi-Fi association: The node connects to a configured local network using a pre-provisioned SSID and password.
- 2) Time synchronization: The internal clock is synchronized with an NTP server. An accurate wall-clock time is required both for meaningful measurement timestamps and for the validation of TLS certificates during the subsequent MQTT handshake.
- 3) Sensor initialization: GPIO pin modes are set and sensor-specific configuration (e.g. DHT11 configuration) is applied.
- 4) MQTT setup: The MQTT client is configured (see Section IV-C).

Once startup is complete, the node enters its measurement loop:

- The MQTT connection is continuously validated and kept alive; on disconnect, the client transparently reconnects.
- Every t milliseconds a full sensor reading is taken across all attached sensors with a default waiting period of $t = 1000$ ms. Values smaller than one second might not change much, as the sensors often have a minimum sampling rate of 1 Hz or lower.
- After n consecutive readings the buffered samples are aggregated into a single record, serialized as JSON and published to the configured MQTT topic. The standard sample aggregation size is $n = 5$, guaranteeing a balance between responsiveness and noise reduction.

Aggregation reduces sensor noise and the influence of isolated outliers. It is performed per field according to its data type: the most recent timestamp of the buffer is taken as the record timestamp, numerical values (analog readings) are averaged arithmetically, and digital boolean signals are decided by majority voting over the buffered samples.

For the KY-028 module, the raw 12-bit ADC reading is first converted to a resistance and then mapped to an absolute temperature in degrees Celsius using the Steinhart-Hart equation [18].

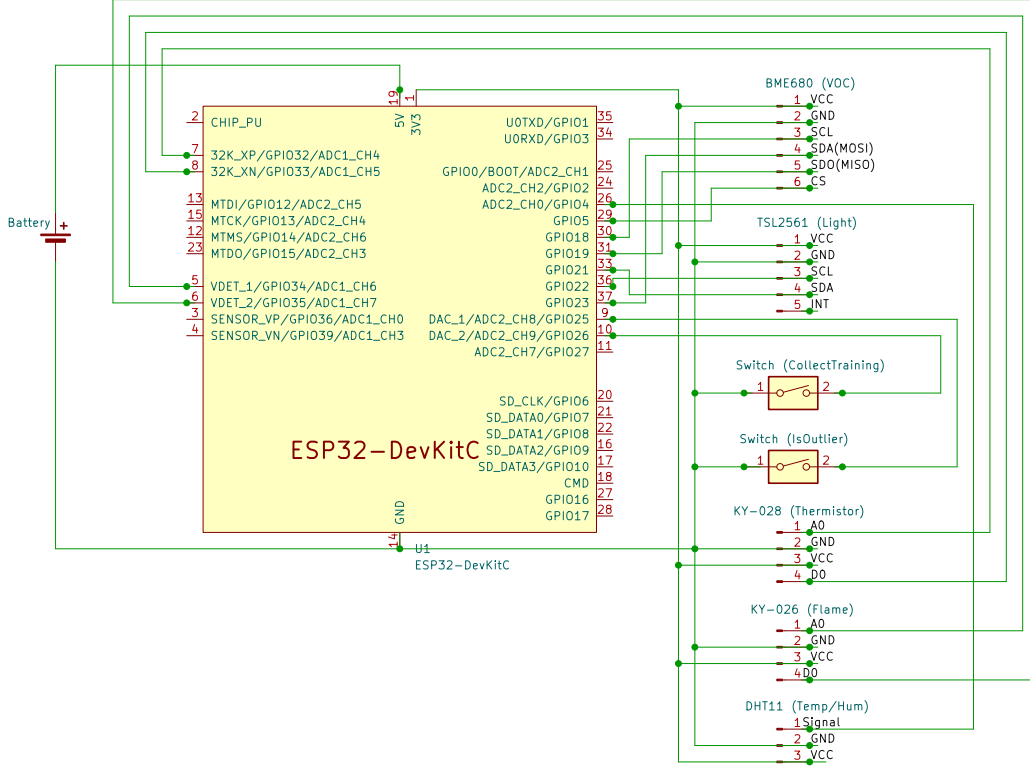


Figure 1. Circuit diagram of the sensor node, showing the logical wiring between the ESP32-DevKitC, sensors, the two auxiliary switches and the power supply, together with their respective GPIO assignments.

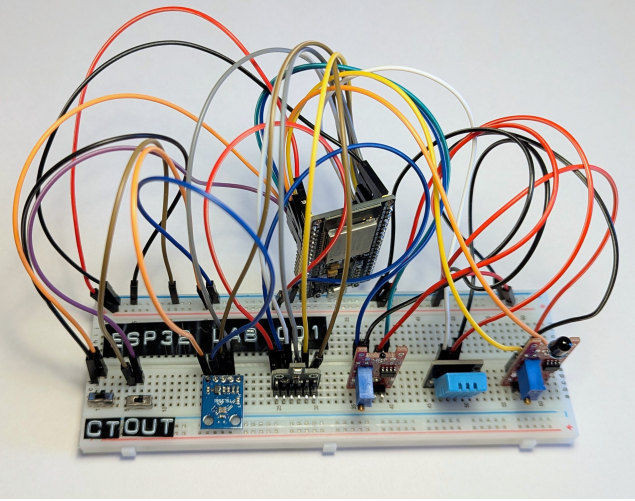


Figure 2. Real physical sensor node with the assembled sensors, the ESP32-DevKitC and the auxiliary switches.

C. MQTT Publication to AWS IoT Core

The sensor node communicates with AWS IoT Core via the MQTT protocol over TLS.

To establish a secure connection to the AWS IoT broker on port 8883, the node requires the broker address and a set of

cryptographic credentials. The publicly available AWS Root CA certificate alongside the individual device certificate and device private key are used for mutual TLS authentication.

The MQTT topic structure is designed around a unique device identifier (e.g., esp32-lab-001). Data exchange uses three primary topics: sensiq/<identifier>/data for publishing the aggregated sensor records, sensiq/<identifier>/test for verifying connectivity, and sensiq/<identifier>/commands reserved for receiving subsequent control commands. Listing 1 shows an example of the JSON payload that the ESP32 node publishes to the data topic after each aggregation interval.

V. AWS SERVICES

Detailed description of the cloud-side processing pipeline. Two functional routes are exposed by the backend: a low-latency live route and a long-term history route. Each is described in its own subsection together with the AWS services it relies on.

A. Live Route

The live route provides a low-latency data delivery mechanism, exposing a `POST /live` endpoint via API Gateway. It returns the most recent sensor reading as a structured JSON payload. The pipeline spans from hardware ingestion through serverless transformation to a managed NoSQL store.

```

1 {
2   "running_time":131147,
3   "timestamp":"2026-06-15T10:45:16Z",
4   "device_id":"esp32-lab-001",
5   "location":"Lab A, OTH Amberg-Weiden, 92224 Amberg,
6     Germany",
7   "dht_humidity":61.6,
8   "dht_temperature":22.6,
9   "dht_heat_index":22.52377,
10  "flame_analog":1089,
11  "flame_digital":false,
12  "thermistor_analog":2178,
13  "thermistor_digital":false,
14  "thermistor_temp":27.95632,
15  "bme_heated_up":false,
16  "bme_temperature":24.89887,
17  "bme_humidity":59.40022,
18  "bme_pressure":964.558,
19  "bme_altitude":413.5006,
20  "bme_voc":97.3798,
21  "tsl_lux":167.8,
22  "is_outlier":false,
23  "collect_training":false
24 }

```

Listing 1. Published JSON payload by the ESP32 sensor node via MQTT.

1) *Data Ingestion and Transformation*: Sensor readings are assembled on the ESP-32 microcontroller and published via MQTT to AWS IoT Core [19]. An IoT topic rule forwards each incoming message to the `handleTransformValidate` Lambda function. This function is responsible for validating and transforming the payload to the internal data format, and classifies outlier readings against predefined thresholds, as described in Section VI-A. Regardless of classification, the validated record is written to a DynamoDB table [20] (`LiveDataDB`) via a `PutItem` operation, which always reflects the latest device state.

2) *Live Data Retrieval*: Incoming `POST /live` requests are routed through the API Gateway to the `handleLiveData` Lambda function. This function performs a read against `LiveDataDB` and returns the latest sensor record as a JSON response. Because the data is served directly from DynamoDB with single-digit millisecond read latency, the live route consistently responds well within the 29-second API Gateway timeout, typically in under one second. The handler returns an error response when the entry in `LiveDataDB` is older than 60 seconds.

B. History Route

The history route provides a long-term data retrieval mechanism, exposing a `POST /history` endpoint via API Gateway. It accepts query parameters such as time ranges, limits, aggregation precision and a device ID, and returns precision-adjusted aggregated historical sensor data as a structured JSON payload. The pipeline bridges data ingestion from AWS IoT Core to a highly optimized query backend.

1) *Data Ingestion and Storage*: Incoming JSON messages from AWS IoT Core are captured by an IoT rule and forwarded to Amazon Kinesis Data Firehose [21]. To mitigate the small-files problem, Firehose buffers the data for up to 15 minutes or until a size of 128 MB is reached before converting it to Apache Parquet format. The Parquet files are written to a S3 Standard data bucket [22], partitioned by year, month, and day to ensure efficient storage and rapid retrieval. S3 lifecycle management policies [23] are configured to retain the data securely for an extended period, while an AWS Glue Data Catalog maintains the structural metadata of the `sensor_data` table. IAM policies tightly control access to the Glue catalog and underlying S3 buckets [24].

2) *Query Execution and Processing*: A dedicated AWS Lambda function coordinates the fulfillment of history requests. It parses the incoming query parameters, validates them, and constructs an Athena query using query execution parameters [25] to defend against SQL attacks. Athena utilizes Glue’s partition projection mapping [26] to prune directories rapidly without needing to load external metadata. A dedicated Athena workgroup [27] manages the query execution, saving the result datasets to a secondary S3 query results bucket.

During execution, the Lambda function continually polls the query execution status [28]. Once the query yields a terminal state, the Lambda handler retrieves the generated results from the S3 bucket, formats them into a JSON response, and returns them to the API Gateway. Although Athena queries induce higher latency compared to the live route, processing is typically completed well within the 29-second timeout limit, mostly in under 5 seconds.

VI. USER INTERACTION

Describes how end users consume the data and receive alerts.

A. Email Notifications via SNS

Explain the alert pipeline: threshold evaluation in the validation Lambda, publication to an SNS topic, and email delivery to subscribed recipients. Cover debounce behaviour and message content.

B. Web Frontend

Describe the React-based dashboard hosted on AWS Amplify: live tiles, historical charts, sensor status indicators, and authentication via AWS Cognito. Include a screenshot or mock-up (see template: figure*).

VII. EVALUATION

A. Testability

The cloud system’s testability is ensured through rigorous automated checks for both infrastructure and application logic. Local yarn scripts automate test execution and coverage reporting.

1) *AWS Infrastructure*: The AWS cloud infrastructure configuration is automatically validated using built-in AWS Cloud Development Kit (CDK) assertions, leveraging the `Template` and `Match` constructs. This ensures that the generated resources and policies match the intended architectural design prior to deployment.

2) *AWS Lambda Handlers*: AWS Lambda functions are thoroughly unit-tested using Python's `unittest` or `pytest` frameworks, employing exhaustive mocking to isolate execution behavior. To evaluate Lambda handlers independently of the API Gateway, a locally defined `TestEvent` JSON payload is utilized to replicate realistic incoming requests during testing. Additionally, for runtime observance and debugging in the deployed environment, Lambda function executions and errors are continually logged to dedicated AWS CloudWatch log groups.

3) *Sensor Node Firmware*: Firmware unit testing utilizes the AUnit framework [29] to cover modules like sensor readings, data aggregation, configurations, and Wi-Fi. Exhaustive MQTT mocking was omitted due to real-world deviations. Instead, the node subscribes to an MQTT test topic during operation to directly verify connection health and publish functionality.

B. Cost Estimation

1) *AWS Service Costs*: The figures in Table I reflect a monthly cost estimate based on an ESP-32 publishing one message every five seconds (roughly 518 000 messages per month). AWS IoT Core is the absolute dominant cost driver. Because each payload is evaluated against two IoT Core rules, AWS bills three units per publish, resulting in 1 554 000 billed message units. This accounts for over 75% of the total monthly cost (\$1.62).

Other minor expenses stem from Kinesis Data Firehose applying a 5 KB minimum billing unit—inflating the logical data volume to 2.59 GB (\$0.08 per month with no free tier)—and Athena scanning costs for an estimated 660 history queries per month. Storage, routing, and alert notification costs remain trivial.

The table contrasts the *Free Tier* expenditure of a new AWS account against the ongoing *Post Free Tier* cost once initial allowances expire. Overall, the architecture operates highly cost-efficiently, totalling approximately \$1.88 and \$2.13 per node per month, respectively.

2) *Hardware Cost*: The one-time hardware cost per sensor node is based on the components used for prototyping. A breakdown of the individual component costs is presented in Table II.

C. Discussion

Currently, the API Gateway Access Key is stored as an environment variable in the React frontend and sent in the header of API requests. This approach is not ideal for security, as it exposes the key in the client-side code and can lead to unauthorized access if the key is compromised. In future iterations, we plan to leverage AWS Cognito for user authentication and authorization.

Operating costs for AWS services could be further reduced by decreasing the publishing frequency from every 5 seconds to a more conservative interval, such as every 30 seconds. This would substantially lower recurring expenses; however, it would compromise real-time monitoring objectives, as critical events might not be detected promptly. Reduced publishing frequency is therefore only viable in scenarios with relaxed real-time requirements.

If the publishing frequency is decreased, implementing a Deep Sleep mode for the ESP32 microcontroller would be a logical next step to significantly extend battery life. However, this is only viable with longer measurement intervals (e.g., every 30 seconds or more). During high-frequency measurements, Deep Sleep could disrupt MQTT connectivity and the sensor reading lifecycle, as the ESP32 is entirely unreachable while in this mode.

Configuring recipients for warning notifications requires manual intervention via the AWS Dashboard. Future iterations should integrate this functionality directly into the React frontend, though this enhancement would require an additional AWS Lambda function designed to update an Amazon DynamoDB table, allowing end-users to seamlessly add or remove notification recipients for specific devices.

Warning thresholds should ideally be configurable on a per-device basis to accommodate varying environmental conditions and requirements. Though, at this stage, thresholds are hardcoded in the Lambda function for simplicity. Using a DynamoDB table to store device-specific thresholds would allow for dynamic updates without code changes.

As demonstrated in [3], integrating machine learning models into Indoor Air Quality (IAQ) monitoring systems can significantly enhance their capabilities by enabling predictive analytics or advanced anomaly detection. A promising enhancement for this architecture involves deploying an outlier detection model on AWS. This could be achieved by utilizing an AWS Lambda function to process the live data stream in real time, identifying anomalies based on an Amazon SageMaker model pre-trained on historical data. This anomaly-detection could complement the existing threshold-based alerting mechanism, providing an additional layer of safety and data insight.

VIII. SUMMARY AND FUTURE WORK

Recap the contribution of the report: a working, MCU-based IoT architecture for environmental monitoring with secure ingestion, dual-path persistence, and a user-facing dashboard. Outline next steps: extended sensor coverage, predictive analytics on historical data, multi-tenant support, and hardening for production deployment.

REFERENCES

- [1] Claes Ahlneck and George Zografi. "The molecular basis of moisture effects on the physical and chemical stability of drugs in the solid state". In: *International journal of pharmaceutics* 62.2-3 (1990), pp. 87–95.

Table I
COST ESTIMATION FOR AWS SERVICES

Service Details	Usage/Quantity	Free Tier	Post Free Tier
IoT Core [19]	518k msgs, 1554k billed	~\$1.62	~\$1.62
Lambda – handleTransformValidate [30]	518k inv, 128 MB / 200 ms	\$0.00	~\$0.06
Lambda – handleLiveData [30]	228k inv, 128 MB / 200 ms	\$0.00	~\$0.04
Lambda – handleHistoryData [30]	660 inv, 128 MB / 200 ms	\$0.00	~\$0.00
DynamoDB [20]	518k WCU, 228k RCU, storage	~\$0.01	~\$0.08
S3 (Parquet + Query Results) [31]	~0.5 GB stored	~\$0.01	~\$0.02
S3 (React Frontend) [31]	Minimal storage used	\$0.00	\$0.00
Firehose [21]	518k msgs, 2.59GB billed	~\$0.08	~\$0.08
Athena [32]	660 queries × 50 MB = 33 GB scanned	~\$0.16	~\$0.16
API Gateway [33]	229k requests	\$0.00	~\$0.08
SNS [34]	Minimal alert volume	\$0.00	~\$0.01
Total		~\$1.88	~\$2.13

Table II
HARDWARE COST PER SENSOR NODE

Component	Unit Cost
ESP32-DevKitC	8 €
Bosch BME680	18 €
TSL2561 Light Sensor	5 €
DHT11 Sensor	3 €
Joy-it KY-026 Flame Sensor	3 €
Joy-it KY-028 Thermistor	3 €
Prototyping equipment	~3 €
Total	~43 €

- [2] Norbert Kockmann, Philipp Thenée, Christoph Fleischer-Trebes, Gabriele Laudadio, and Timothy Noël. “Safety assessment in development and operation of modular continuous-flow processes”. In: *Reaction Chemistry & Engineering* 2.3 (2017), pp. 258–280.
- [3] Rafia Mumtaz, Syed Mohammad Hassan Zaidi, Muhammad Zeeshan Shakir, Uferah Shafi, Muhammad Moez Malik, Ayesha Haque, Sadaf Mumtaz, and Syed Ali Raza Zaidi. “Internet of things (Iot) based indoor air quality sensing and predictive analytic—A COVID-19 perspective”. In: *Electronics* 10.2 (2021), p. 184.
- [4] Gonçalo Marques and Rui Pitarma. “An internet of things-based environmental quality management system to supervise the indoor laboratory conditions”. In: *Applied Sciences* 9.3 (2019), p. 438.
- [5] Simon Völkl, Johannes Schieder, Sebastian Rosner, and Andre Tien Vu. *Sensiq GitHub Repository*. [Online]. URL: <https://github.com/s-voelkl/BCN-2026SSTeamMagenta-Sensiq>.
- [6] BerryBase. *ESP32 NodeMCU Development Board*. [Online]. URL: <https://www.berrybase.de/en/esp32-nodemcu-development-board>.
- [7] Espressif Systems. *ESP32-WROOM-32 Datasheet*. [Online]. URL: https://documentation.espressif.com/esp32-wroom-32-datasheet_en.pdf.
- [8] Asair. *DHT11 Temperature and Humidity Module*. [Online]. URL: <https://asairsensors.com/product/dht11-sensor/>.
- [9] Joy-It. *KY-026 Analog Flame Sensor*. [Online]. URL: <https://sensorkit.joy-it.net/en/sensors/ky-026>.
- [10] Joy-It. *KY-028 Temperature Sensor (Thermistor)*. [Online]. URL: <https://sensorkit.joy-it.net/en/sensors/ky-028>.
- [11] BerryBase. *TSL2561 Digital Light / Brightness Sensor Datasheet*. [Online]. URL: <https://www.berrybase.de/en/product-datasheet/019234a424aa7020aee21ad30f4ed8c8/create>.
- [12] Adafruit. *Adafruit TSL2561 Light Sensor Driver*. [Online]. URL: https://github.com/adafruit/Adafruit_TSL2561.
- [13] Bosch Sensortec. *BME680 Gas, Pressure, Temperature and Humidity Sensor Datasheet*. [Online]. URL: <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme680-ds001.pdf>.
- [14] Random Nerd Tutorials. *ESP32: BME680 Environmental Sensor using Arduino IDE*. [Online]. URL: <https://randomnerdtutorials.com/esp32-bme680-sensor-arduino/>.
- [15] Adafruit. *Adafruit BME680 Library*. [Online]. URL: https://github.com/adafruit/Adafruit_BME680.
- [16] KiCad Services Corporation. *KiCad PCB Design Suite*. Version 10.0.3. 2026. URL: <https://www.kicad.org>.
- [17] Arduino. *Arduino Language Reference*. [Online]. URL: <https://www.arduino.cc/reference/en/>.
- [18] John S Steinhart and Stanley R Hart. “Calibration curves for thermistors”. In: *Deep sea research and oceanographic abstracts*. Vol. 15. 4. Elsevier. 1968, pp. 497–503.
- [19] Amazon. *AWS IoT Core Documentation*. [Online]. URL: <https://docs.aws.amazon.com/iot/>.
- [20] Amazon. *Amazon DynamoDB Documentation*. [Online]. URL: <https://docs.aws.amazon.com/dynamodb/>.
- [21] Amazon Web Services. *Amazon Kinesis Data Firehose Documentation*. [Online]. URL: <https://docs.aws.amazon.com/firehose/>.
- [22] Amazon Web Services. *Amazon S3 Storage Classes*. [Online]. URL: <https://aws.amazon.com/de/s3/storage-classes/>.
- [23] Amazon Web Services. *Managing the lifecycle of objects in Amazon S3*. [Online]. URL: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html>.
- [24] Amazon Web Services. *Control Access to Data Catalogs with IAM Policies in Amazon Athena*. [Online]. URL: <https://docs.aws.amazon.com/athena/latest/ug/datacatalogs-iam-policy.html>.
- [25] Amazon Web Services. *Use execution parameters in Amazon Athena*. [Online]. URL: <https://docs.aws.amazon.com/athena/latest/ug/querying-with-prepared-statements-querying-using-execution-parameters.html>.
- [26] Amazon Web Services. *Use Partition Projection with Amazon Athena*. [Online]. URL: <https://docs.aws.amazon.com/athena/latest/ug/partition-projection.html>.
- [27] Amazon Web Services. *Use Workgroups to Control Query Access and Costs in Amazon Athena*. [Online]. URL: <https://docs.aws.amazon.com/athena/latest/ug/workgroups-manage-queries-control-costs.html>.

- [28] Amazon Web Services. *Query Execution Status in Amazon Athena*. [Online]. URL: https://docs.aws.amazon.com/athena/latest/APIReference/API_QueryExecutionStatus.html.
- [29] Brian T. Park. *AUnit Testing Framework for Arduino*. Version 1.7.1. 2026. URL: <https://github.com/bxparks/AUnit>.
- [30] Amazon. *AWS Lambda Documentation*. [Online]. URL: <https://docs.aws.amazon.com/lambda/>.
- [31] Amazon. *Amazon Simple Storage Service (S3) Documentation*. [Online]. URL: <https://docs.aws.amazon.com/s3/>.
- [32] Amazon. *Amazon Athena Documentation*. [Online]. URL: <https://docs.aws.amazon.com/athena/>.
- [33] Amazon. *Amazon API Gateway Documentation*. [Online]. URL: <https://docs.aws.amazon.com/apigateway/>.
- [34] Amazon. *Amazon Simple Notification Service (SNS) Documentation*. [Online]. URL: <https://docs.aws.amazon.com/sns/>.